

Practical – 3

Name:- Aniruddh Jirange

PRN:- 202501040076

Division:- CS1 (C14)

3.1.1. Numpy Array Operations

Write a Python program to create a NumPy array based on user input and display the following:

- The created NumPy array.
- The number of dimensions of the array using ndim
- The shape of the array using shape
- The total number of elements in the array using size

Assume all input elements are valid integers.

Input Format:

- The first line contains two space-separated integers, r and c , representing the number of rows and the number of columns.
- The next r lines contain c space-separated integers representing the elements of the array, entered row by row.

Output Format:

- The created NumPy array (printed as a 2D array).
- An integer representing the number of dimensions of the array.
- A tuple representing the shape of the array.
- An integer representing the total number of elements in the array.

Note: Use `reshape()` function to reshape the input array with the specified number of rows and columns.

Code:

```
import numpy as np
r,c = map(int,input().split())
elements = []
for _ in range(r):
    row = list(map(int,input().split()))
    elements.append(row)

arr = np.array(elements).reshape(r,c)

print(arr)
print(arr.ndim)
print(arr.shape)
print(arr.size)
```

Average time

0.019 s

19.50 ms



Maximum time

0.049 s

49.00 ms



✓ 2 out of 2 shown test case(s) passed

✓ 10 out of 10 hidden test case(s) passed

✓ Test case 1 49 ms

Debug



Expected output

Actual output

3 4

3 4

1 2 3 4

1 2 3 4

5 6 7 8

5 6 7 8

9 10 11 12

9 10 11 12

[[·1·2·3·4]

[[·1·2·3·4]

·[·5·6·7·8]

·[·5·6·7·8]

·[·9·10·11·12]]

·[·9·10·11·12]]

2

2

(3,·4)

(3,·4)

12

12

✓ Test case 2 8 ms

Debug



Expected output

Actual output

0 0

0 0

[]

[]

2

2

(0,·0)

(0,·0)

0

0

3.2.1. Numpy: Matrix Operations

The given code takes two 3 x 3 matrices, `matrix_a` , and `matrix_b` , as input from the user and converts them into NumPy arrays.

Task:

You are required to compute and display the results of the following matrix operations:

1. **Addition** (`matrix_a + matrix_b`)
2. **Subtraction** (`matrix_a - matrix_b`)
3. **Element-wise Multiplication** (`matrix_a * matrix_b`)
4. **Matrix Multiplication** (`matrix_a . matrix_b`)
5. **Transpose of Matrix A**

Input Format:

- The user will input 3 rows for `matrix_a` , each containing 3 integers separated by spaces.
- Similarly, the user will input 3 rows for `matrix_b`, each containing 3 integers separated by spaces.

Output Format:

The program should display the results of the operations in the following order:

1. The result of Addition.
2. The result of Subtraction.
3. The result of Element-wise Multiplication.
4. The result of Matrix Multiplication.
5. The Transpose of Matrix A.

Code:

```
import numpy as np

# Input matrices
print("Enter Matrix A:")
matrix_a = np.array([list(map(int, input().split())) for i in range(3)])

print("Enter Matrix B:")
matrix_b = np.array([list(map(int, input().split())) for i in range(3)])

# Addition
addition_result = matrix_a + matrix_b
print("Addition (A + B):")
print(addition_result)

# Subtraction
subtraction_result = matrix_a - matrix_b
print("Subtraction (A - B):")
print(subtraction_result)

# Multiplication (element-wise)
elementwise_multiplication_result = matrix_a * matrix_b
print("Element-wise Multiplication (A * B):")
print(elementwise_multiplication_result)

# Matrix multiplication (dot product)
```

```
matrix_multiplication_result = np.dot(matrix_a, matrix_b)
print("A dot B:")
print(matrix_multiplication_result)
```

Transpose

```
transpose_a = matrix_a.T
print("Transpose of A:")
print(transpose_a)
```

Average time
0.048 s
47.75 ms

Maximum time
0.092 s
92.00 ms

✓ 2 out of 2 shown test case(s) passed
✓ 2 out of 2 hidden test case(s) passed

✓ Test case 1 92 ms

Debug

Expected output	Actual output
Enter Matrix-A:	Enter Matrix-A:
1 2 3	1 2 3
4 5 6	4 5 6
7 8 9	7 8 9
Enter Matrix-B:	Enter Matrix-B:
1 1 1	1 1 1
1 1 1	1 1 1
1 1 1	1 1 1
Addition (A+B):	Addition (A+B):
[[2 3 4]]	[[2 3 4]]
[- 5 -6 -7]	[- 5 -6 -7]
[- 8 -9 -10]	[- 8 -9 -10]
Subtraction (A-B):	Subtraction (A-B):
[[0 1 2]]	[[0 1 2]]
[- 3 -4 -5]	[- 3 -4 -5]
[- 6 -7 -8]	[- 6 -7 -8]

Element-wise Multiplication (A.*B):	Element-wise Multiplication (A.*B):
[[1 2 3]	[[1 2 3]
· [4 5 6]	· [4 5 6]
· [7 8 9]]	· [7 8 9]]
A·dot·B:	A·dot·B:
[[·6··6··6]	[[·6··6··6]
· [15 15 15]	· [15 15 15]
· [24 24 24]]	· [24 24 24]]
Transpose of A:	Transpose of A:
[[1 4 7]	[[1 4 7]
· [2 5 8]	· [2 5 8]
· [3 6 9]]	· [3 6 9]]

✓ Test case 2 35 ms  Debug  

Expected output	Actual output
Enter Matrix A:	Enter Matrix A:
2 4 8	2 4 8
9 6 5	9 6 5
7 8 9	7 8 9
Enter Matrix B:	Enter Matrix B:
10 12 13	10 12 13
14 15 16	14 15 16
17 18 19	17 18 19
Addition (A.+B):	Addition (A.+B):
[[12 16 21]	[[12 16 21]
· [23 21 21]	· [23 21 21]
· [24 26 28]]	· [24 26 28]]
Subtraction (A.-B):	Subtraction (A.-B):
[[·8··8··5]	[[·8··8··5]
· [·5··9·-11]	· [·5··9·-11]
· [-10·-10·-10]]	· [-10·-10·-10]]

Element-wise Multiplication (A.*B):	Element-wise Multiplication (A.*B):
[[-20 -48 104]	[[-20 -48 104]
· [126 -90 -80]	· [126 -90 -80]
· [119 144 171]]	· [119 144 171]]
A·dot·B:	A·dot·B:
[[212 228 242]	[[212 228 242]
· [259 288 308]	· [259 288 308]
· [335 366 390]]	· [335 366 390]]
Transpose of A:	Transpose of A:
[[2 9 7]	[[2 9 7]
· [4 6 8]	· [4 6 8]
· [8 5 9]]	· [8 5 9]]

3.2.2. Numpy: Horizontal and Vertical Stacking of Arrays

You are given two arrays `arr1` and `arr2`. You need to perform horizontal and vertical stacking operations on them using NumPy.

- **Horizontal Stacking:** Stack the two matrices horizontally (side by side).
- **Vertical Stacking:** Stack the two matrices vertically (one below the other).

Input Format:

- The program should first prompt the user to input two 3x3 arrays.
- Each array consists of 3 rows, and each row contains 3 space-separated integers.
- The user will input the two arrays row by row.

Output Format:

- The program should display the result of the Horizontal Stack (side-by-side stacking) of the two arrays.
- The program should then display the result of the Vertical Stack (one below the other) of the two arrays.

Code:

```
import numpy as np
```

```
# Input matrices
```

```
print("Enter Array1:")
```

```
arr1 = np.array([list(map(int, input().split())) for i in range(3)])
```

```
print("Enter Array2:")
```

```
arr2 = np.array([list(map(int, input().split())) for i in range(3)])
```

```
# Perform horizontal stacking (hstack)
```

```
a=np.hstack((arr1,arr2))
```

```
print("Horizontal Stack:")
```

```
print(a)
```

```
# Perform vertical stacking (vstack)
```

```
b=np.vstack((arr1,arr2))
```

```
print("Vertical Stack:")
```

```
print(b)
```

Average time

0.039 s

39.25 ms



Maximum time

0.060 s

60.00 ms

**✓ 2 out of 2 shown test case(s) passed****✓ 2 out of 2 hidden test case(s) passed****✓ Test case 1** 60 ms **Debug**

Expected output

Enter Array1:

1 2 3**4 5 6****7 8 9**

Enter Array2:

4 5 6**7 8 9****10 11 12**

Horizontal Stack:

[[-1 -2 -3 -4 -5 -6]

- [-4 -5 -6 -7 -8 -9]

- [-7 -8 -9 -10 -11 -12]]

Vertical Stack:

[[-1 -2 -3]

- [-4 -5 -6]

- [-7 -8 -9]

- [-4 -5 -6]

- [-7 -8 -9]

- [10 -11 -12]]

Actual output

Enter Array1:

1 2 3**4 5 6****7 8 9**

Enter Array2:

4 5 6**7 8 9****10 11 12**

Horizontal Stack:

[[-1 -2 -3 -4 -5 -6]

- [-4 -5 -6 -7 -8 -9]

- [-7 -8 -9 -10 -11 -12]]

Vertical Stack:

[[-1 -2 -3]

- [-4 -5 -6]

- [-7 -8 -9]

- [-4 -5 -6]

- [-7 -8 -9]

- [10 -11 -12]]

✓ Test case 2 32 ms **Debug**

Expected output

Enter Array1:

-1 -2 -3**-4 -5 -6****-7 -8 -9**

Enter Array2:

10 11 12**13 14 15****16 17 18**

Horizontal Stack:

[[-1 -2 -3 -10 -11 -12]

- [-4 -5 -6 -13 -14 -15]

- [-7 -8 -9 -16 -17 -18]]

Vertical Stack:

[[-1 -2 -3]

- [-4 -5 -6]

- [-7 -8 -9]

- [10 -11 -12]

- [13 -14 -15]

- [16 -17 -18]]

Actual output

Enter Array1:

-1 -2 -3**-4 -5 -6****-7 -8 -9**

Enter Array2:

10 11 12**13 14 15****16 17 18**

Horizontal Stack:

[[-1 -2 -3 -10 -11 -12]

- [-4 -5 -6 -13 -14 -15]

- [-7 -8 -9 -16 -17 -18]]

Vertical Stack:

[[-1 -2 -3]

- [-4 -5 -6]

- [-7 -8 -9]

- [10 -11 -12]

- [13 -14 -15]

- [16 -17 -18]]

3.2.3. Numpy: Custom Sequence Generation

Write a Python program that takes the following inputs from the user:

- Start value: The starting point of the sequence.
- Stop value: The sequence should end before this value.
- Step value: The increment between each number in the sequence.

The program should then generate a sequence using numpy based on these inputs and print the generated sequence.

Input Format:

- The user will input three integer values: start, stop, and step, each on a new line.

Output Format:

- The program should print the generated sequence based on the input values.

Code:

```
import numpy as np

# Take user input for the start, stop, and step of the sequence
start = int(input())
stop = int(input())
step = int(input())

# Generate the sequence using np.arange()
a = np.arange(start, stop, step)

# Print the generated sequence
print(a)
```

Average time

0.025 s

25.00 ms



Maximum time

0.050 s

50.00 ms



✓ 2 out of 2 shown test case(s) passed

✓ 2 out of 2 hidden test case(s) passed

✓ Test case 1 50 ms

Debug



Expected output

Actual output

3

3

10

10

2

2

[3·5·7·9]

[3·5·7·9]

✓ Test case 2 17 ms

Debug



Expected output

Actual output

10

10

20

20

30

30

[10]

[10]

3.2.4. Numpy: Arithmetic and Statistical Operations, Mathematical Operations, Bitwise

You are given two arrays A and B. Your task is to complete the function `array_operations`, which will convert these lists into NumPy arrays and perform the following operations:

1. Arithmetic Operations:

- Compute the element-wise sum, difference, and product of the two arrays.

2. Statistical Operations:

- Calculate the mean, median, and standard deviation of array A.

3. Bitwise Operations:

- Perform bitwise AND, bitwise OR, and bitwise XOR on the arrays (ex: $A_i \text{ OR } B_i$).

Input Format:

- The first line contains space-separated integers representing the elements of array A.
- The second line contains space-separated integers representing the elements of array B.

Output Format:

- For each operation (arithmetic, statistical, and bitwise), print the results in the specified format as shown in sample test cases.

Code:

```
import numpy as np
```

```
def array_operations(A, B):
```

```
    # Convert A and B to NumPy arrays
```

```
    A = np.array(A)
```

```
    B = np.array(B)
```

```
    # Arithmetic Operations
```

```
    sum_result = A + B
```

```
    diff_result = A - B
```

```
    prod_result = A * B
```

```
    # Statistical Operations
```

```
    mean_A = np.mean(A)
```

```
    median_A = np.median(A)
```

```
    std_dev_A = np.std(A)
```

```
    # Bitwise Operations
```

```
    and_result = A & B
```

```
    or_result = A | B
```

```
    xor_result = A ^ B
```

```
    # Output results with one space between each element
```

```
    print("Element-wise Sum:", ' '.join(map(str, sum_result)))
```

```
    print("Element-wise Difference:", ' '.join(map(str, diff_result)))
```

```
print("Element-wise Product:", ' '.join(map(str, prod_result)))
```

```
print(f"Mean of A: {mean_A}")
```

```
print(f"Median of A: {median_A}")
```

```
print(f"Standard Deviation of A: {std_dev_A}")
```

```
print("Bitwise AND:", ' '.join(map(str, and_result)))
```

```
print("Bitwise OR:", ' '.join(map(str, or_result)))
```

```
print("Bitwise XOR:", ' '.join(map(str, xor_result)))
```

```
A = list(map(int, input().split())) # Elements of array A
```

```
B = list(map(int, input().split())) # Elements of array B
```

```
array_operations(A, B)
```

Average time

0.020 s

19.67 ms



Maximum time

0.031 s

31.00 ms



✓ 1 out of 1 shown test case(s) passed

✓ 2 out of 2 hidden test case(s) passed



Test case 1

31 ms

Debug



Expected output

1 2 3 4

5 6 7 8

Element-wise Sum: -6-8-10-12

Element-wise Difference: - -4- -4- -4- -4

Element-wise Product: -5-12-21-32

Mean of A: -2.5

Median of A: -2.5

Standard Deviation of A: -1.118033988749895

Bitwise AND: -1-2-3-0

Bitwise OR: -5-6-7-12

Bitwise XOR: -4-4-4-12

Actual output

1 2 3 4

5 6 7 8

Element-wise Sum: -6-8-10-12

Element-wise Difference: - -4- -4- -4- -4

Element-wise Product: -5-12-21-32

Mean of A: -2.5

Median of A: -2.5

Standard Deviation of A: -1.118033988749895

Bitwise AND: -1-2-3-0

Bitwise OR: -5-6-7-12

Bitwise XOR: -4-4-4-12

3.2.5. Numpy: Copying and Viewing Arrays

The given code takes a list of integers as input and converts it into a NumPy array. Your task is to complete the code by:

- Creating a view of the `original_array` and assigning it to `view_array`.
- Creating a copy of the `original_array` and assigning it to `copy_array`.

After completing these steps, observe how modifying the view affects the `original_array`, while modifying the copy does not.

Input Format:

- A single line of space-separated integers.

Output Format:

- After modifying the view:

Original array after modifying view: **<original_array>**

View array: **<view_array>**

- After modifying the copy:

Original array after modifying copy: **<original_array>**

Copy array: **<copy_array>**

Code:

```
import numpy as np
```

```
inputlist = list(map(int,input().split(" ")))
```

```
# Original array
```

```
original_array = np.array(inputlist)
```

```
# Create a view
```

```
view_array = original_array.view()
```

```
# Create a copy
```

```
copy_array = original_array.copy()
```

```
# Modify the view
```

```
view_array[0] = 99
```

```
print("Original array after modifying view:", original_array)
```

```
print("View array:", view_array)
```

```
# Modify the copy
```

```
copy_array[1] = 88
```

```
print("Original array after modifying copy:", original_array)
```

```
print("Copy array:", copy_array)
```

Average time

0.015 s

15.25 ms

Maximum time

0.036 s

36.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

36 ms

Debug

Expected output

Actual output

10 20 30 40 50 60 70 80

10 20 30 40 50 60 70 80

Original array after modifying view: [99 20 30 40 50 60 70 80]

Original array after modifying view: [99 20 30 40 50 60 70 80]

View array: [99 20 30 40 50 60 70 80]

View array: [99 20 30 40 50 60 70 80]

Original array after modifying copy: [99 20 30 40 50 60 70 80]

Original array after modifying copy: [99 20 30 40 50 60 70 80]

Copy array: [10 88 30 40 50 60 70 80]

Copy array: [10 88 30 40 50 60 70 80]

Test case 2

11 ms

Debug

Expected output

Actual output

99 2 3 4 5

99 2 3 4 5

Original array after modifying view: [99 2 3 4 5]

Original array after modifying view: [99 2 3 4 5]

View array: [99 2 3 4 5]

View array: [99 2 3 4 5]

Original array after modifying copy: [99 2 3 4 5]

Original array after modifying copy: [99 2 3 4 5]

Copy array: [99 88 3 4 5]

Copy array: [99 88 3 4 5]

3.2.6. Numpy: Searching, Sorting, Counting, Broadcasting

The given code in the editor takes a single array, `array1`, as space-separated integers as input from the user.

Additionally, it takes the following inputs:

- `search_value`: The value to search for in the array.
- `count_value`: The value to count its occurrences in the array.
- `broadcast_value`: The value to add for broadcasting across the array.

You need to complete the code to perform the following operations:

1. **Searching**: Find the indices where `search_value` appears in `array1` and print these indices.
2. **Counting**: Count how many times `count_value` appears in `array1` and print the count.
3. **Broadcasting**: Add `broadcast_value` to each element of `array1` using broadcasting, and print the resulting array.
4. **Sorting**: Sort `array1` in ascending order and print the sorted array.

Input Format:

1. A single line containing space-separated integers representing `array1`.
2. An integer `search_value` represents the value to search for in the array.
3. An integer `count_value` represents the value to count in the array.
4. An integer `broadcast_value` represents the value to add to each element of the array.

Output Format:

1. The indices where `search_value` occurs in `array1`.

2. The count of occurrences of count_value in array1.
3. The array after adding the broadcast_value to each element.
4. The sorted array.

Code:

```
import numpy as np

# Input array from the user
array1 = np.array(list(map(int, input().split()))))

# Searching
search_value = int(input("Value to search: "))
count_value = int(input("Value to count: "))
broadcast_value = int(input("Value to add: "))

# Find indices where value matches in array1
a=np.where(array1==search_value)[0]
print(a)

# Count occurrences in array1
b=np.count_nonzero(array1==count_value)
print(b)

# Broadcasting addition
c=array1 + broadcast_value
print(c)

# Sort the first array
d=np.sort(array1)
print(d)
```

Average time
0.029 s
28.75 ms



Maximum time
0.041 s
41.00 ms



✓ 2 out of 2 shown test case(s) passed

✓ 2 out of 2 hidden test case(s) passed

✓ Test case 1 41 ms

Debug





Expected output	Actual output
1 1 1 2 2 2	1 1 1 2 2 2
Value to search: 1	Value to search: 1
Value to count: 2	Value to count: 2
Value to add: 2	Value to add: 2
[0 · 1 · 2]	[0 · 1 · 2]
3	3
[3 · 3 · 3 · 4 · 4 · 4]	[3 · 3 · 3 · 4 · 4 · 4]
[1 · 1 · 1 · 2 · 2 · 2]	[1 · 1 · 1 · 2 · 2 · 2]

✓ Test case 2 30 ms

Debug





Expected output	Actual output
1 2 3 4 5	1 2 3 4 5
Value to search: 6	Value to search: 6
Value to count: 6	Value to count: 6
Value to add: 6	Value to add: 6
[]	[]
0	0
[· 7 · · 8 · · 9 · 10 · 11]	[· 7 · · 8 · · 9 · 10 · 11]
[1 · 2 · 3 · 4 · 5]	[1 · 2 · 3 · 4 · 5]

3.2.7. Student Data Analysis and Operations

Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations:

- **Print all student details:** Display the complete details of all students, including roll numbers and marks for all subjects.
- **Find total students:** Determine the total number of students in the dataset.
- **Print all student roll numbers:** Extract and print the roll numbers of all students.
- **Print Subject 1 marks:** Extract and print the marks of all students in Subject 1.
- **Find minimum marks in Subject 2:** Identify the lowest marks in Subject 2.
- **Find maximum marks in Subject 3:** Identify the highest marks in Subject 3.
- **Print all subject marks:** Display the marks of all students for each subject.
- **Find total marks of students:** Compute the total marks for each student across all subjects.
- **Find the average marks of each student:** Compute the average marks for each student.
- **Find average marks of each subject:** Compute the average marks for all students in each subject.
- **Find average marks of Subject 1 and Subject 2:** Compute the average marks for Subject 1 and Subject 2.
- **Find average marks of Subject 1 and Subject 3:** Compute the average marks for Subject 1 and Subject 3.
- **Find the roll number of the student with maximum marks in Subject 3:** Identify the student with the highest marks in Subject 3 and print their roll number.

- **Find the roll number of the student with minimum marks in Subject 2:** Identify the student with the lowest marks in Subject 2 and print their roll number.
- **Find the roll number of students who scored 24 marks in Subject 2:** Identify students who obtained exactly 24 marks in Subject 2 and print their roll numbers.
- **Find the count of students who got less than 40 marks in Subject 1:** Count the number of students who scored less than 40 marks in Subject 1.
- **Find the count of students who got more than 90 marks in Subject 2:** Count the number of students who scored more than 90 marks in Subject 2.
- **Find the count of students who scored ≥ 90 in each subject:** Count the number of students who scored 90 or more marks in each subject.
- **Find the count of subjects in which each student scored ≥ 90 :** Determine how many subjects each student scored 90 or more marks in.
- **Print Subject 1 marks in ascending order:** Sort and print the marks of students in Subject 1 in ascending order.
- **Print students who scored between 50 and 90 in Subject 1:** Display students who scored marks between 50 and 90 in Subject 1.
- **Find index positions of students who scored 79 in Subject 1:** Identify the index positions of students who scored exactly 79 marks in Subject 1.

Note: Fill in the missing code to perform the above-mentioned operations.

Code:

```
import numpy as np
```

```
a = np.loadtxt("Sample.csv", delimiter=',', skiprows=1)
```

```
# 1
```

```
print("All student Details:\n", a)
```

```
# 2
```

```
print("Total Students:", len(a))
```

```
# 3
```

```
print("All Student Roll Nos", a[:, 0])
```

```
# 4
```

```
print("Subject 1 Marks", a[:, 1])
```

```
# 5
```

```
print("Min marks in Subject 2", np.min(a[:, 2]))
```

```
# 6
```

```
print("Max marks in Subject 3", np.max(a[:, 3]))
```

```
# 7
```

```
print("All subject marks:", a[:, 1:])
```

```
# 8
```

```
total = a[:,1] + a[:,2] + a[:,3]
print("Total Marks", total)
```

```
# 9
```

```
print(np.round(total / 3, 1))
```

```
# 10
```

```
avg = np.round(np.mean(a[:,1:], axis=0), 1)
print("Average Marks of each subject", avg)
```

```
# 11
```

```
print("Average Marks of S1 and S2", avg[:2])
```

```
# 12
```

```
print("Average Marks of S1 and S3", avg[[0,2]])
```

```
# 13
```

```
print("Roll no who got maximum marks in Subject 3",
a[np.argmax(a[:,3]), 0])
```

```
# 14
```

```
print("Roll no who got minimum marks in Subject 2",
a[np.argmin(a[:,2]), 0])
```

```
# 15
```

```
print("Roll no who got 24 marks in Subject 2", a[a[:,2] ==
24][:,0].reshape(-1,1))
```

16

```
print("Count of students who got marks in Subject 1 < 40",  
      np.sum(a[:,1] < 40))
```

17

```
print("Count of students who got marks in Subject 2 > 90:",  
      np.sum(a[:,2] > 90))
```

18

```
print("Count of students in each subject who got marks >= 90:",  
      np.sum(a[:,1:] >= 90, axis=0))
```

19

```
print("Roll no:", a[:,0])  
  
print("Count of subjects in which student got marks >= 90:",  
      np.sum(a[:,1:] >= 90, axis=1))
```

20

```
print(np.sort(a[:,1]))
```

21

```
print(a[(a[:,1] >= 50) & (a[:,1] <= 90)])  
print(a)
```

22 (FINAL LINE — STOP HERE)

```
print(np.where(a[:,1] == 79))
```

Average time
0.050 s
50.00 ms



Maximum time
0.050 s
50.00 ms



✓ 1 out of 1 shown test case(s) passed

✓ Test case 1

50 ms

Debug



Expected output

Actual output

All student Details:

All student Details:

- [[301. - 67. - 77. - 88.]]

- [[301. - 67. - 77. - 88.]]

- [302. - 78. - 88. - 77.]]

- [302. - 78. - 88. - 77.]]

- [303. - 45. - 56. - 89.]]

- [303. - 45. - 56. - 89.]]

- [304. - 88. - 98. - 45.]]

- [304. - 88. - 98. - 45.]]

- [305. - 78. - 88. - 99.]]

- [305. - 78. - 88. - 99.]]

- [306. - 68. - 78. - 36.]]

- [306. - 68. - 78. - 36.]]

- [307. - 79. - 12. - 45.]]

- [307. - 79. - 12. - 45.]]

- [308. - 46. - 45. - 77.]]

- [308. - 46. - 45. - 77.]]

- [309. - 89. - 32. - 88.]]

- [309. - 89. - 32. - 88.]]

- [310. - 79. - 24. - 33.]]

- [310. - 79. - 24. - 33.]]

Total Students: 10

Total Students: 10

All Student Roll Nos: [301. 302. 303. 304. 305. 306. 307. 308. 309. 310.]

All Student Roll Nos: [301. 302. 303. 304. 305. 306. 307. 308. 309. 310.]

Subject 1 Marks: [67. 78. 45. 88. 78. 68. 79. 46. 89. 79.]

Subject 1 Marks: [67. 78. 45. 88. 78. 68. 79. 46. 89. 79.]

Min marks in Subject 2 12.0

Min marks in Subject 2 12.0

Max marks in Subject 3 99.0

Max marks in Subject 3 99.0

Max marks in Subject 3 99.0

Max marks in Subject 3 99.0

All subject marks: - [[67. 77. 88.]]

All subject marks: - [[67. 77. 88.]]

- [78. 88. 77.]]

- [78. 88. 77.]]

- [45. 56. 89.]]

- [45. 56. 89.]]

- [88. 98. 45.]]

- [88. 98. 45.]]

- [78. 88. 99.]]

- [78. 88. 99.]]

- [68. 78. 36.]]

- [68. 78. 36.]]

- [79. 12. 45.]]

- [79. 12. 45.]]

- [46. 45. 77.]]

- [46. 45. 77.]]

- [89. 32. 88.]]

- [89. 32. 88.]]

- [79. 24. 33.]]

- [79. 24. 33.]]

Total Marks: [232. 243. 190. 231. 265. 182. 136. 168. 209. 136.]

Total Marks: [232. 243. 190. 231. 265. 182. 136. 168. 209. 136.]

[77.3 81. - 63.3 77. - 88.3 60.7 45.3 56. - 69.7 45.3]

[77.3 81. - 63.3 77. - 88.3 60.7 45.3 56. - 69.7 45.3]

Average Marks of each subject: [71.7 59.8 67.7]

Average Marks of each subject: [71.7 59.8 67.7]

Average Marks of S1 and S2: [71.7 59.8]

Average Marks of S1 and S2: [71.7 59.8]

Average Marks of S1 and S3: [71.7 67.7]

Average Marks of S1 and S3: [71.7 67.7]

Roll no who got maximum marks in Subject 3 305.0

Roll no who got maximum marks in Subject 3 305.0

Roll no who got minimum marks in Subject 2 307.0

Roll no who got minimum marks in Subject 2 307.0

Roll no who got 24 marks in Subject 2: [[310.]]

Roll no who got 24 marks in Subject 2: [[310.]]

Count of students who got marks in Subject 1 < 40: 0	Count of students who got marks in Subject 1 < 40: 0
Count of students who got marks in Subject 2 > 90: 1	Count of students who got marks in Subject 2 > 90: 1
Count of students in each subject who got marks >= 90: [0 1 1]	Count of students in each subject who got marks >= 90: [0 1 1]
Roll no: [301, 302, 303, 304, 305, 306, 307, 308, 309, 310.]	Roll no: [301, 302, 303, 304, 305, 306, 307, 308, 309, 310.]
Count of subjects in which student got marks >= 90: [0 0 0 1 1 0 0 0 0 0]	Count of subjects in which student got marks >= 90: [0 0 0 1 1 0 0 0 0 0]
[45, 46, 67, 68, 78, 78, 79, 79, 88, 89.]	[45, 46, 67, 68, 78, 78, 79, 79, 88, 89.]
[[301, 67, 77, 88.]	[[301, 67, 77, 88.]
· [302, 78, 88, 77.]	· [302, 78, 88, 77.]
· [304, 88, 98, 45.]	· [304, 88, 98, 45.]
· [305, 78, 88, 99.]	· [305, 78, 88, 99.]
· [306, 68, 78, 36.]	· [306, 68, 78, 36.]
· [307, 79, 12, 45.]	· [307, 79, 12, 45.]
· [309, 89, 32, 88.]	· [309, 89, 32, 88.]
· [310, 79, 24, 33.]]	· [310, 79, 24, 33.]]
[[301, 67, 77, 88.]	[[301, 67, 77, 88.]
· [302, 78, 88, 77.]	· [302, 78, 88, 77.]
· [303, 45, 56, 89.]	· [303, 45, 56, 89.]
· [304, 88, 98, 45.]	· [304, 88, 98, 45.]
· [305, 78, 88, 99.]	· [305, 78, 88, 99.]
· [306, 68, 78, 36.]	· [306, 68, 78, 36.]
· [307, 79, 12, 45.]	· [307, 79, 12, 45.]
· [308, 46, 45, 77.]	· [308, 46, 45, 77.]
· [309, 89, 32, 88.]	· [309, 89, 32, 88.]
· [310, 79, 24, 33.]]	· [310, 79, 24, 33.]]